# YOUTUBE CHANNEL ANALYTICS

Business Intelligence & Analytics Project

---

**YouTube Data API v3 • Python • PostgreSQL • SQL • Power BI • VS Code**

End-to-end real-data analytics solution for the Regaltos YouTube channel

**Channel:** Regaltos
**Handle:** @soulregaltos9810

Prepared as a GitHub portfolio project

# 1. Executive Summary

This project delivers an end-to-end YouTube Channel Analytics solution using real YouTube data. The workflow is YouTube Data API v3 → Python → PostgreSQL → SQL → Power BI, with VS Code used as the development environment.

The supplied final dashboard provides KPI cards, interactive Published Year, Published Month and Video Title filters, top-video rankings, yearly and monthly trends, engagement analysis and a channel information panel.

The supplied dashboard shows approximately **1,729 videos**, **652M views**, **58M likes**, **700K comments**, **377.06K average views per video**, **2.48M subscribers** and a channel creation date of **11 Dec 2015**. These values are taken from the supplied dashboard image.

# 2. Objectives

| Objective | Implementation |
|---|---|
| Collect real data | YouTube Data API v3. |
| Automate extraction | Python scripts from VS Code. |
| Store data | PostgreSQL. |
| Analyze data | Reusable SQL queries. |
| Build dashboard | Power BI with DAX and interactive slicers. |
| Portfolio delivery | GitHub README, source code, PBIX, screenshot and report. |

# 3. Data Architecture

```
YouTube Data API v3
↓
Python extraction
↓
PostgreSQL
↓
SQL analysis
↓
Power BI model
↓
Interactive Dashboard
```

| Layer | Technology | Purpose |
|---|---|---|
| Source | YouTube Data API v3 | Channel/video metadata and statistics. |
| Development | VS Code | Python and SQL development. |
| Extraction | Python | API calls, transformation and loading. |
| Database | PostgreSQL | Persistent analytical storage. |
| Analytics | SQL | KPIs, rankings and time analysis. |
| BI | Power BI | Interactive reporting. |

# 4. API Key Security

The YouTube API key is private and must never be placed directly in Python code, SQL, README.md, PBIX documentation or GitHub. The project uses a root .env file and python-dotenv.

```
# .env
YOUTUBE_API_KEY=your_private_key_here
```

```
from dotenv import load_dotenv
import os

load_dotenv()
API_KEY = os.getenv("YOUTUBE_API_KEY")
```

The real .env file should remain private. GitHub should contain only a safe .env.example placeholder.

# 5. VS Code Setup

```
pip install google-api-python-client python-dotenv
```

# 6. Project Repository Structure

```
youtube-channel-analytics/
■■■ python/
■ ■■■ channel_data.py
■ ■■■ video_data.py
■■■ sql/
■ ■■■ schema.sql
■ ■■■ analysis.sql
■ ■■■ views.sql
■■■ powerbi/
■ ■■■ youtube_analytics_dashboard.pbix
■■■ screenshots/
■ ■■■ youtube_analytics_dashboard.png
■■■ documentation/
■ ■■■ YouTube_Analytics_Project_Report.pdf
■■■ .env.example
■■■ .gitignore
■■■ README.md
```

# 7. PostgreSQL Data Layer

| Logical table | Purpose |
|---|---|
| public_channels | Channel ID, name, subscribers and channel metadata. |
| public_videos | Video ID, title, published_at, views, likes and comments. |
| Analytical SQL views | Reusable summaries consumed by Power BI. |

# 8. Core SQL Queries

## Total videos

```
SELECT COUNT(DISTINCT video_id) AS total_videos
FROM public_videos;
```

## Total views, likes and comments

```
SELECT SUM(views) AS total_views,
SUM(likes) AS total_likes,
SUM(comments) AS total_comments
FROM public_videos;
```

## Average views per video

```
SELECT AVG(views) AS avg_views_per_video
FROM public_videos;
```

## Top 10 videos by views

```
SELECT title, views, likes, comments
FROM public_videos
ORDER BY views DESC
LIMIT 10;
```

# 9. Year & Month Analysis SQL

```
SELECT EXTRACT(YEAR FROM published_at) AS published_year,
COUNT(DISTINCT video_id) AS total_videos,
SUM(views) AS total_views
FROM public_videos
GROUP BY EXTRACT(YEAR FROM published_at)
ORDER BY published_year;

SELECT EXTRACT(MONTH FROM published_at) AS published_month,
COUNT(DISTINCT video_id) AS total_videos,
SUM(views) AS total_views
FROM public_videos
GROUP BY EXTRACT(MONTH FROM published_at)
ORDER BY published_month;
```

# 10. Engagement SQL

```
SELECT title, views, likes, comments,
CASE WHEN views > 0
THEN ROUND((likes::numeric / views) * 100, 2)
ELSE 0 END AS like_rate_percentage
FROM public_videos
ORDER BY views DESC
LIMIT 10;
```

# 11. Power BI DAX Measures

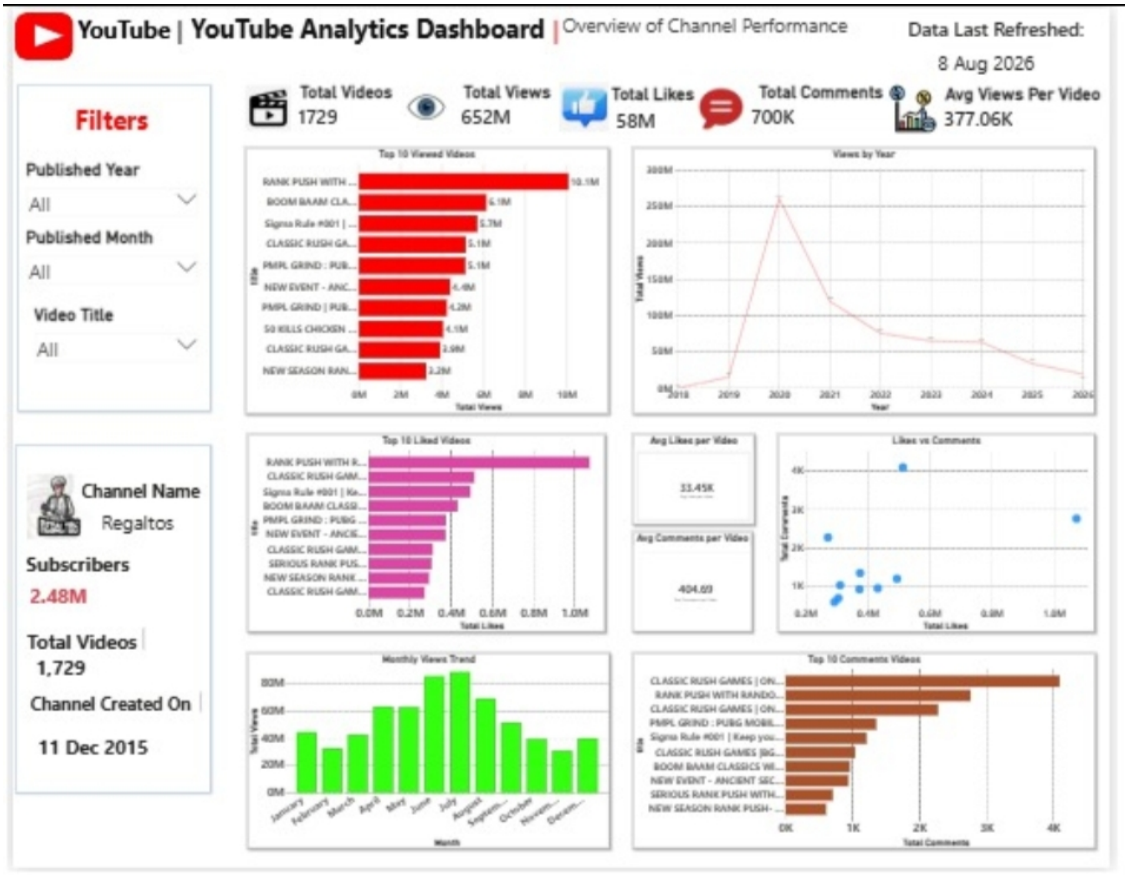
```
Total Videos =
DISTINCTCOUNT('public videos'[video_id])

Total Views =
SUM('public videos'[views])

Total Likes =
SUM('public videos'[likes])

Total Comments =
SUM('public videos'[comments])

Avg Views per Video =
DIVIDE([Total Views], [Total Videos])
```

# 12. Interactive Filters

**Published Year:** year-level filtering. **Published Month:** January-to-December analysis. **Video Title:** focused individual-video analysis.

# 13. Final Power BI Dashboard

The following is the supplied dashboard screenshot used as the final project evidence:




The dashboard contains KPI cards, filters, Top 10 Viewed Videos, Views by Year, Top 10 Liked Videos, Avg Likes per Video, Likes vs Comments, Monthly Views Trend, Top 10 Comments Videos and a Channel Name panel.

# 14. Dashboard Visuals & Purpose

| Visual | Purpose |
|---|---|
| Total Videos | Unique video count. |
| Total Views | Overall reach. |
| Total Likes | Total positive engagement. |
| Total Comments | Audience discussion volume. |
| Avg Views per Video | Average reach per video. |
| Top 10 Viewed Videos | Highest-reach content. |
| Views by Year | Long-term trend. |
| Top 10 Liked Videos | Most-liked content. |
| Likes vs Comments | Engagement relationship. |
| Monthly Views Trend | Month-level performance. |
| Top 10 Comments Videos | Discussion leaders. |
| Channel Panel | Channel context and identity. |

# 15. Key Dashboard Evidence

| Metric | Supplied dashboard |
|---|---|
| Total Videos | 1,729 |
| Total Views | 652M |
| Total Likes | 58M |
| Total Comments | 700K |
| Avg Views per Video | 377.06K |
| Subscribers | 2.48M |
| Channel Created On | 11 Dec 2015 |
| Data Last Refreshed | 8 Aug 2026 |

# 16. Refresh & Reproducibility

```
YouTube API
↓
Python extraction
↓
PostgreSQL update
↓
SQL analysis
↓
Power BI refresh
↓
Updated dashboard
```

This workflow allows the dashboard to be refreshed from updated source data rather than relying on manually entered dashboard numbers.

# 17. GitHub Security

Never commit the actual API key or .env file. Use a placeholder file for documentation.

```
# .gitignore
.env
*.key
*.pem
__pycache__/
.venv/
```

# 18. GitHub Upload Checklist

| Upload | Status / purpose |
|---|---|
| README.md | Project overview and setup instructions. |
| Python scripts | API extraction and processing. |
| SQL scripts | Database and analysis logic. |
| PBIX file | Power BI dashboard, if sharing is appropriate. |
| Dashboard PNG | Portfolio preview. |
| PDF report | Professional project documentation. |
| .env.example | Safe API-key configuration template. |
| Real .env / API key | DO NOT upload. |

**Project:** YouTube Channel Analytics Dashboard • **Channel:** Regaltos • **Stack:** YouTube Data API v3 → Python → PostgreSQL → SQL → Power BI

---

API credentials are intentionally excluded from this report and must remain private.